- The controller object's `flash`, `headers`, `logger`, `params`, `request`, `response`, and `session` are available as accessor methods in the view. Apart from the flash, view code probably shouldn't use these directly, because the responsibility for handling them should rest with the controller. However, we do find this useful when debugging. For example, the following `html.erb` template uses the `debug` method to display the contents of the session, the details of the parameters, and the current response:

```
<h4>Session</h4>  <%= debug(session) %>
<h4>Params</h4>   <%= debug(params) %>
<h4>Response</h4> <%= debug(response) %>
```

- The current controller object is accessible using the attribute named `controller`. This allows the template to call any public method in the controller (including the methods in `ActionController::Base`).

- The path to the base directory of the templates is stored in the attribute `base_path`.

What Goes in a Template

Out of the box, Rails supports two types of templates:

- ERB templates are a mixture of content and embedded Ruby. They're typically used to generate HTML pages.

- Jbuilder[106] templates generate JSON responses.

By far, the one that you'll be using the most will be ERB. In fact, you made extensive use of ERB templates in developing the Depot application.

So far in this chapter, we've focused on producing output. In Chapter 21, *Action Dispatch and Action Controller*, we focused on processing input. In a well-designed application, these two are not unrelated: the output we produce contains forms, links, and buttons that guide the end user to producing the next set of inputs. As you might expect by now, Rails provides a considerable amount of help in this area too.